2. Warehouse-scale Computers (WSC): Features & Challenges

Operating Systems (OS)

Master in Innovation and Research in Informatics (MIRI)

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC)

2025/2026 Q1

FIB

Contents

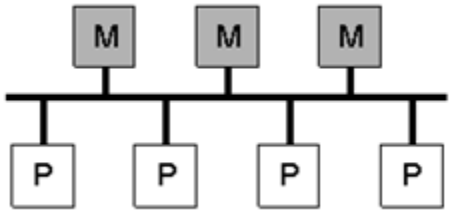
- **HW architecture of WSCs**
- SW workloads in WSCs
- Operating system for WSCs

Hardware architecture of WSCs

- Let's consider systems with multiple CPUs, which can be organized in different ways

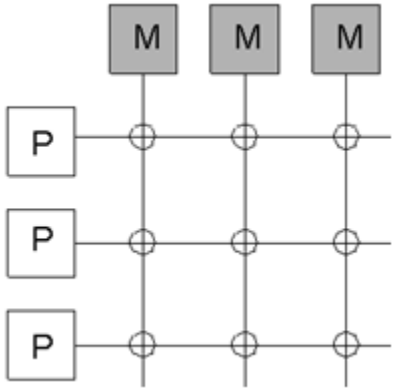
A. Hardware enables shared memory (there is a single address space): e.g. multiprocessors

1. Uniform Memory Access (UMA)



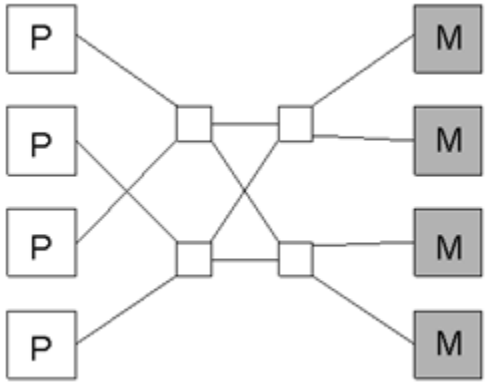
a) bus-based

↓ scalability



b) crossbar-based

↓ cost



c) switch-based

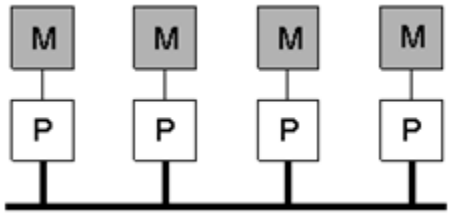
↓ latency

Hardware architecture of WSCs

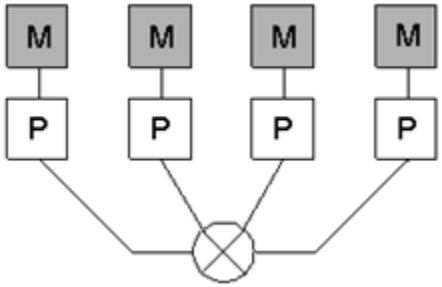
2. Non-Uniform Memory Access (NUMA)

- Some memory is associated with each CPU
- Quick access to local memory, slower to the rest

a) bus-based



b) switch-based



- ➢ Data placement is critical for performance
- ➢ Performance could be improved by adding caches, but introducing memory coherence issues
  - – You studied cache coherence protocols for multiprocessors in the BS degree

Hardware architecture of WSCs

B. Every computer has its own private memory
   - Computers must be connected through an **interconnection network**
     - Communication is through message passing
   - We will focus on massive multicomputer systems
     - Consisting of thousands of individual computing nodes connected together to act as a single larger computer
   - Architecture consists of three primary subsystems:
     1. Computing equipment (computing nodes with their corresponding networking and storage subsystems)
     2. Power distribution system
     3. Cooling infrastructure

} We will cover them in Lesson 6

Hardware architecture of WSCs

- WSCs are built of 1U-shaped low-end server-class computers connected in racks of 40 to 80 units
  - Cost efficiency
    - Trade-off performance and cost
  - e.g. Google (2017): 2 x sockets x 18 Intel Haswell 2.3GHz cores, 256 GB DDR3-1600 DRAM, 2 x SATA HDD each up to 8 TB, or SSD flash drives with 1 TB [HePa17]
- Supercomputers use more powerful processors and lower latency networks



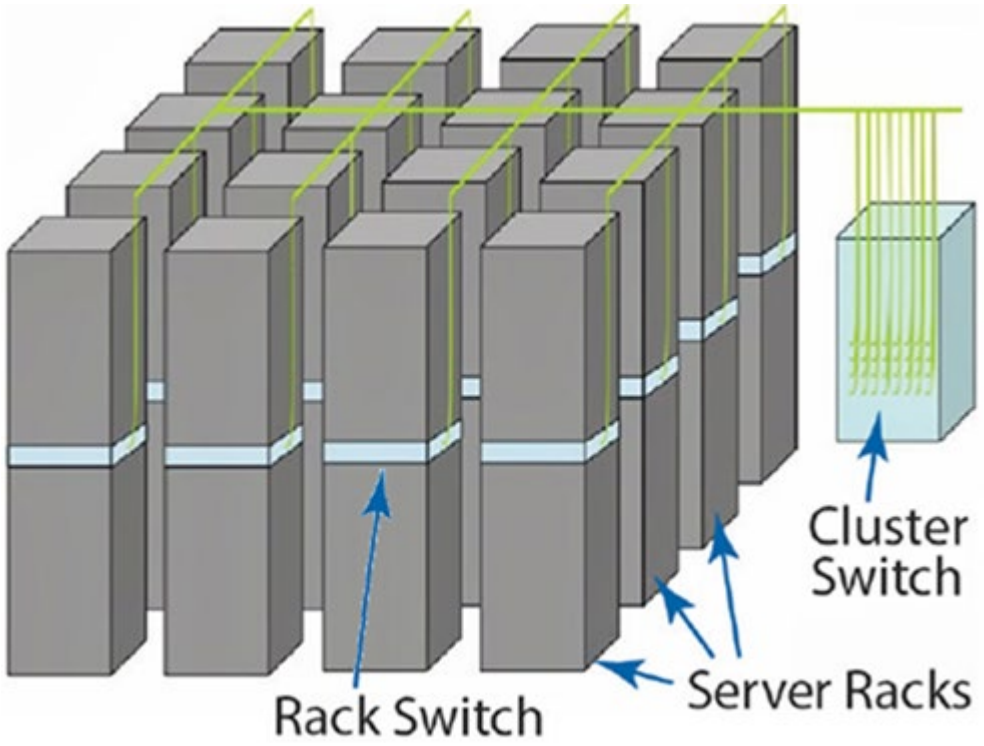
Courtesy of [BaCH13]

Hardware architecture of WSCs

- WSCs use hierarchical switched networks

1. **Rack switches** to communicate within and out of each rack

2. A hierarchy of **cluster switches** connect the arrays of racks



Courtesy of [BaCH13]

Hardware architecture of WSCs

- Gigabit Ethernet is the usual network fabric in WSCs and datacenters
  - Ethernet rack switches are commodity (low cost)
  - Cluster switches are more expensive, but cost can be reduced oversubscribing servers to the uplinks from the rack switch to the cluster switch
- Supercomputers also have high-performance networks (that support RDMA) such as Infiniband
  - Those are lower-latency and higher-bandwidth networks, but also much more expensive
  - Typical HPC applications are more interdependent and communicate more frequently

Hardware architecture of WSCs

- Storage is provided either as:

a) Directly Attached Storage (DAS)

  - Disk drives directly connected to each server and rely on networking for access to data on remote disks (through a distributed file system)

  ↑ Reduced hardware costs

    - Disks can leverage the existing server enclosure

  ↑ Improved network utilization at the servers

    - Network ports are dynamically shared between compute and storage tasks

Hardware architecture of WSCs

- Storage is provided either as:

b) Network Attached Storage (NAS)

  - Dedicated file-serving appliances directly connected to cluster-level switching fabric

  ↑ Simpler deployment

    - Data management responsibilities are separated (and can be outsourced to a NAS vendor)

  ↑ Easier to enforce QoS

    - Storage and compute tasks do not interfere

Hardware architecture of WSCs

- Storage can be accessed using:

a) Unstructured APIs

  - Access data through file-like abstractions
  - e.g., distributed file systems such as Google's File System and Google's Colossus

b) Structured APIs

  - Access data through database-like abstractions
  - There are both SQL and (especially) NoSQL DBs
  - e.g., distributed database systems such as Google's BigTable and Amazon's Dynamo

Complementary reading

- Barroso, L.A., Hölzle, U., Ranganathan, P., The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines, 3rd edition, Morgan and Claypool Publishers, 2018
  - Chapter 1.6: Architectural Overview of WSCs
  - Chapter 3: WSC Hardware Building Blocks

Hardware architecture of WSCs

- Consequences of WSC architecture:

1. Multiple heterogeneous computers
2. Multiple autonomous processes
3. Processes execute concurrently
4. Knowledge on each computer is local
5. Process address space is not shared
6. Only communication is by message passing
7. Network delays in process communication

Hardware architecture of WSCs

- Consequences of WSC architecture:

8. Processes may not always be available (network partitions, process disconnections)
9. Processes can fail independently
10. High probability of failures (it grows with number of components)
11. Increment of energy-related costs
12. Independent (imperfect) clocks (no global time and perfect synchronization unfeasible)

Contents

- HW architecture of WSCs
- **SW workloads in WSCs**
- Operating system for WSCs

Workloads

- Workloads are also relevant, as they pose requirements for the OS, which derive from:

1. Functionality of the WSC expected by the workloads
   A. Ability to communicate with remote entities
      - Need for naming systems to locate entities
      - Need for process communication paradigms
   B. Ability to use (and share) remote data/resources
      - Need for naming systems to locate data/resources
      - Need for coordination and synchronization algorithms to share data/resources safely

Workloads

1. Functionality of the WSC expected by the workloads
   C. Ability to run efficiently on large amounts of compute and storage resources
      - Need for scalable workload processing frameworks
      - Need for efficient cluster management systems
      - Need for distributed data storage systems
   D. Ability to be available and run reliably in the presence of failures
      - Need for fault-tolerant systems without disruption to end-user applications

Workloads

2. Characteristics of the workloads in WSCs
   A. Both interactive and batch processing workloads are present
      - e.g. interactive: web search, social networking, maps
      - e.g. batch: calculate metadata useful to interactive services: create search indexes from web crawls, process satellite images to create map tiles
   B. Variable (resource) requirements due to high workload churn (i.e. rate of change)
      - e.g. changing number of customers, requests
      - e.g. adding/removing services and service instances, upgrading software versions

Workloads

2. Characteristics of the workloads in WSCs
   C. Workloads exhibit ample parallelism, needed to exploit the amount of parallel HW in a WSC
   i. Data-level parallelism: large number of distinct datasets that require independent processing
      - e.g. pages from a web crawl used to create indexes
   ii. Request-level parallelism: large number of requests from independent users to interactive services
      - e.g. web search requests over a (mostly read-only) index

Contents

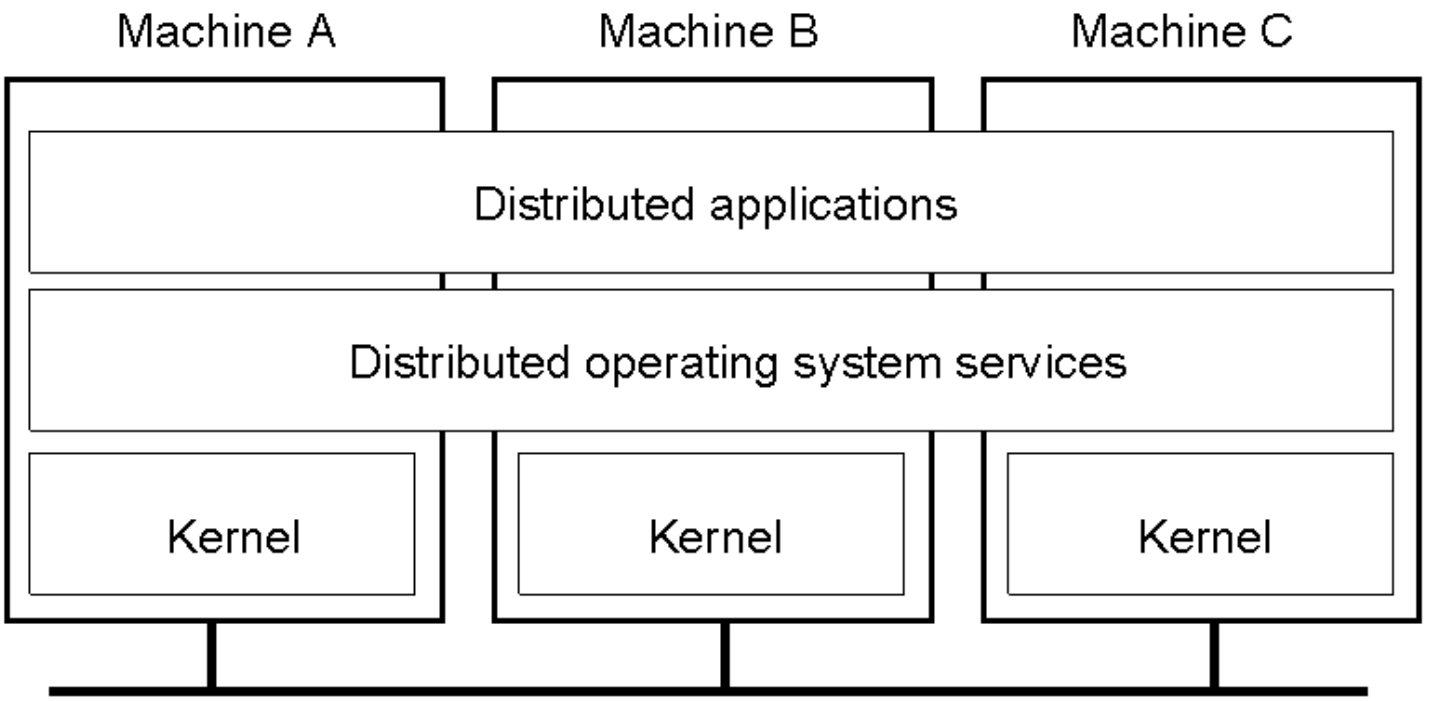
- HW architecture of WSCs
- SW workloads in WSCs
- **Operating system for WSCs**

Operating system for WSCs

- Desktop operating system
  - Acts as a resource manager and enables resource sharing (CPU, memory, I/O devices, ...)
  - Abstracts the hardware by providing a virtual interface that is easier to use
- How can we design an operating system for a WSC?
  - OPTION A: Distributed O.S.
  - OPTION B: Network O.S.
  - OPTION C: Middleware

Distributed operating system

- Networked but homogeneous computers managed as a single system (distribution transparency)

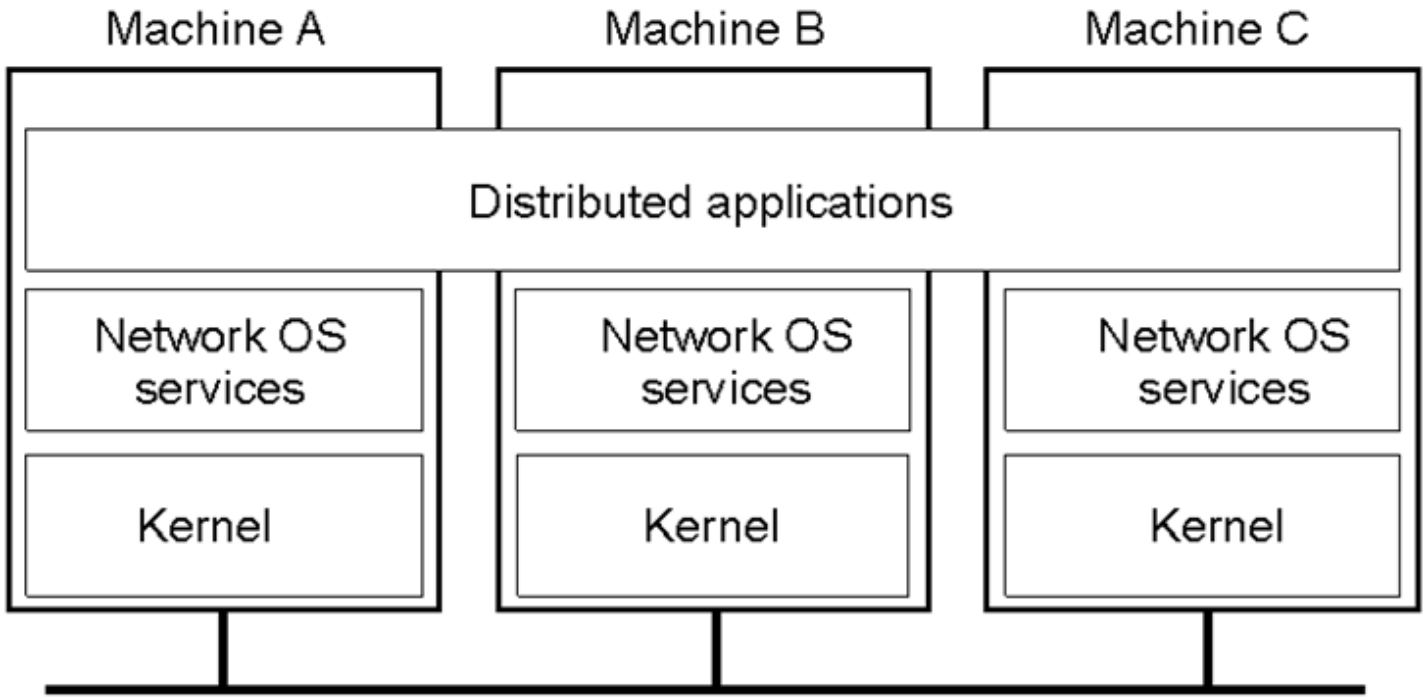


a **single** O.S.

Courtesy of [TaSt02]

Network operating system

- Networked independent computers offer services for **explicit** remote access (lacks transparency)

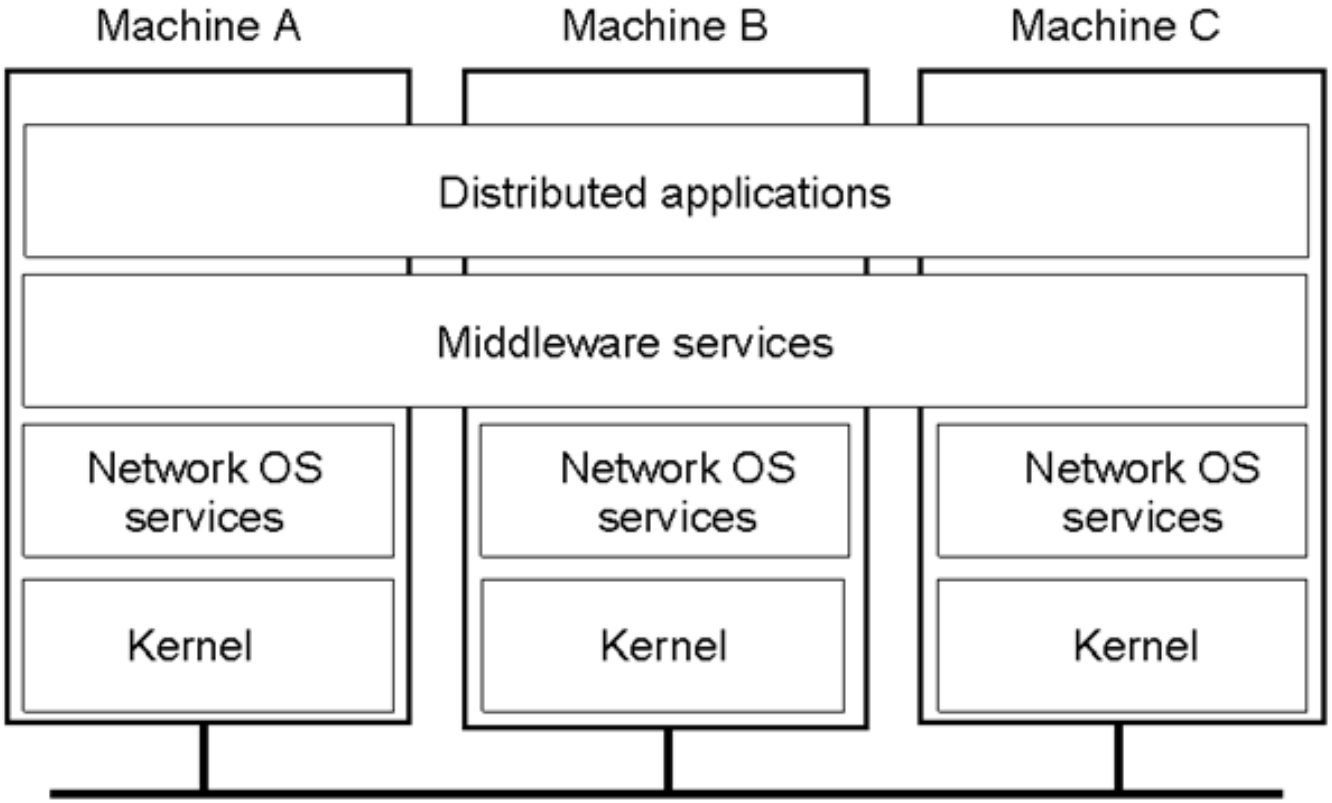


several local O.S.

Courtesy of [TaSt02]

Middleware

- Networked independent computers offer a view of a **single coherent** system (distribution transparency)



several local O.S. + **middleware** services

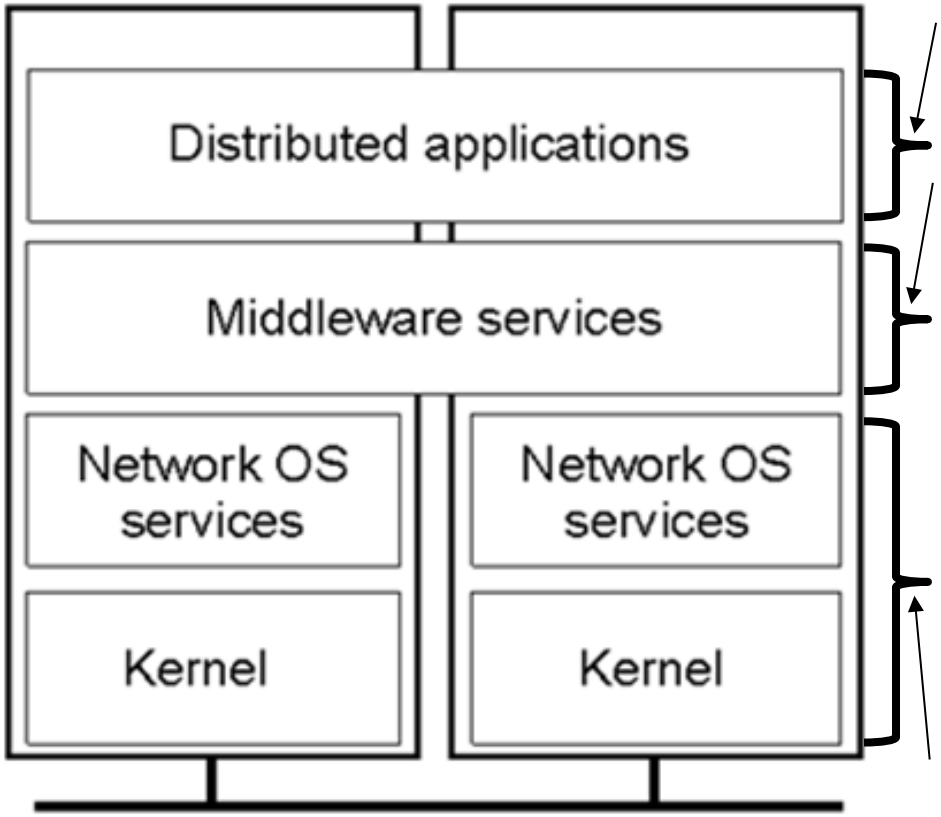
Courtesy of [TaSt02]

Operating system for WSCs

	Distributed OS	Network OS	Middleware
Independent computers	✗	✓	✓
Transparency	✓ ✓	✗	✓
Performance/Scalability	✗	✓ ✓	✓
Usability/Programmability	✓ ✓	✗	✓

Operating system for WSCs

- WSCs have adopted the middleware approach



Application-level SW

Cluster-level infrastructure SW
Manage resources and provide services at cluster level (distributed versions of classic OS features + new features required by the WSC architecture)
⇒ This acts as an OS for the WSC

Platform-level SW
Firmware, kernel, OS distribution & libraries present in individual hosts. Similar to those on a desktop platform.
Virtualization is now popular in WSCs

Contents

- HW architecture of WSCs
- SW workloads in WSCs
- Operating system for WSCs
  - **Middleware services**
  - Cross-cutting objectives

Middleware services

a) **Cluster management and scheduling**

1. Provide basic process management services: tracking machine liveness, starting, monitoring, and terminating tasks
2. Control the mapping of tasks to HW resources (process scheduling) and arbitrate resource allocation
3. Provide mechanisms for process resilience: state machine replication, checkpoint/restore

– e.g. Apache Mesos and Aurora

– e.g. Google's Borg, Kubernetes, and Omega

Middleware services

a) **Cluster management and scheduling**

- Scheduling in WSCs has two dimensions:
  1. Which process must run next?
  2. Where (CPU or host) must the process run?
     - Affinity, data/process migrations, load balancing
- Time-sharing vs. space-sharing
- Processes could be independent or collaborative (group of processes working together)
  - Collaborative processes can benefit from scheduling them together at the same time
  - Also from scheduling them close together in space (i.e. to reduce latencies)

Middleware services

b) **Process communication** paradigms

- Message passing: Sockets, MPI
- Message queuing: through queues (Apache ActiveMQ)
- Remote invocation: RPC, RMI, Web Services
- Group communication (multicast): deliver messages to all processes in a group (JGroups)
- Publish-subscribe: through event propagation (Apache Kafka)
- Shared data spaces: shared storage (JavaSpaces)
- Distributed shared memory (DSM): Treadmarks

Middleware services

c) **Virtualization**

1. Create and destroy virtual machines readily and with little overhead
   a. Scale up/down the number of VMs (elasticity)
   b. Fast service provisioning/removal
2. Improve both security and performance isolation of users' applications
3. Allow agile and fine-grain dynamic resource provisioning (consolidation, load balancing)
4. Simplify fault tolerance using VM snapshots
5. Allow **live migration** of VMs

Middleware services

d) Distributed **data storage** services

- Store efficiently huge amounts of data
- Data items are distributed across multiple servers
  - They can be also replicated
- Items can be stored in-memory or on-disk-storage
- Items can be accessed via un/structured APIs
- Support for atomic distributed transactions
- e.g. distributed shared memory (DSM) systems
- e.g. distributed file systems: GFS, HDFS
- e.g. distributed DB systems: SQL vs. NoSQL: BigTable, Cassandra, Spanner

Middleware services

e) **Replication** services
- Enhance availability, reliability, and performance
- Replicated data should be **consistent**
- How do we transparently and efficiently keep all the replicas up-to-date and consistent?
  - Trade-off **consistency vs. performance**

f) Automated deployment and **maintenance**
1. Image distribution, configuration management
2. Monitoring for system introspection
   a. Application performance monitoring
   b. Platform-level health monitoring

Middleware services

g) Distributed **coordination services**

- Reliable and consistent authoritative source of coordination for codependent middleware services

1. Synchronization algorithms for mutual exclusion based solely on message passing
2. Election algorithms to select a coordinator process (or a primary replica) within a group
3. Naming systems to locate remote entities
4. Consensus algorithms to allow a group of processes to agree on some value

- e.g. Google's Chubby, Yahoo!'s Zookeeper

Middleware services

h) Meaningful **global views** assembly

1. Global time via physical clock synchronization
2. Event ordering by means of logical clocks
3. Global snapshots
   a. Checkpoint/restore fault tolerance
   b. Evaluation of global predicates (garbage collection, deadlock detection, distributed debugging)

i) Programming frameworks (e.g. MapReduce)

– Improve **programmability** by automatically handling data partitioning, task distribution, fault tolerance, and message-based communication

Middleware services

- In this course, we will discuss some of these adapted/new services
  - Process scheduling and resilience (in Lesson 3a)
  - Virtualization (in Lesson 3b)
  - Distributed data storage (in Lessons 4 and 5)
    - In-memory and on-disk-storage
    - DSM systems, file systems, DB systems
    - Replication and consistency
- Some of the rest are presented in the 'Distributed Systems' part of CPDS course

Contents

- HW architecture of WSCs
- SW workloads in WSCs
- Operating system for WSCs
  - Middleware services
  - **Cross-cutting objectives**

Cross-cutting objectives

1. Transparency
   - Ability of a distributed system of presenting itself as a single computer system
   - Applies to several aspects of the system based on which of them are hidden/masked
     - i) access mechanisms to objects, ii) location of objects, iii) migration of objects, iv) concurrent access to objects, v) replication of objects, vi) failure/recovery of objects
   - Transparency may be set as a goal, but fully achieving it is a different story
     - Impossibility to hide latencies and failures completely
     - Trade-off **transparency vs. performance**

Cross-cutting objectives

2. Fault tolerance
   - Ability of the system to meet its specifications in the presence of faults

Type of failure	Description
Crash failure	Process halts and remains halted
Omission failure	Sent message never arrives at the other end
Timing failure	Process response lies outside a specified time interval
Response failure	Process response is incorrect: the value of the response is wrong or the process deviates from the correct flow of control
Arbitrary (Byzantine) failure	Process may arbitrarily omit intended steps or take unintended steps, which exhibit differently to different observers

Cross-cutting objectives

2. Fault tolerance
   - Deal with failures using **redundancy**
   A. Information redundancy
      - Add extra bits to allow for error detection/recovery
        - e.g. parity bits, cyclic redundancy checks (CRC)
   B. Physical redundancy
      - **Replicate** hardware/software components or data items
   C. Time redundancy
      - Repeat an operation several times
        - e.g. retry a remote operation until a reply is received

Cross-cutting objectives

3. Scalability
   - Ability to support the growing in the number resources/users (size scalability)

   A. Partition data and/or computations and distribute them across multiple computers
      - Data-level parallelism
      - Request-level parallelism

   B. Replicate components across the system
      - Trade-off **consistency vs. scalability**

   C. Use of decentralized algorithms
      - There is not any coordinator: Nodes cooperate using only its local data to run the algorithm incrementally

Cross-cutting objectives

4. Energy and cost efficiency
   - Energy awareness and reduction of energy-related costs and environmental impact in all the relevant subsystems in a WSC
     - IT equipment, power supply, and cooling

   A. Capital costs are directly related to:
      - Maximum power that must be supplied
      - Cooling needed to remove the waste heat

   B. Energy usage by all the subsystems determines the electricity bill (i.e. operational costs)

   ⇒ We will focus on this objective in Lesson 6 from a software management perspective

References

[BaCH13] L.A. Barroso, J. Clidaras and U. Hoelzle. 2013. The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines (2nd ed.). Morgan and Claypool Publishers

[HePa17] Hennessy, J.L., Patterson, D.A., Computer Architecture: A Quantitative Approach, 6th Edition, Elsevier, Morgan Kaufmann, 2017

[TaSt02] Andrew S. Tanenbaum and Maarten van Steen. 2002. Distributed Systems: Principles and Paradigms. Prentice-Hall, Inc., Upper Saddle River, NJ, USA